

ALGORITMA DAN STRUKTUR DATA
LATIHAN PRAKTIKU 5 DOUBLE LINKED LIST



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TUGAS PRAKTIKUM ASD – SINGLE LINKED LIST

1. Buatlah double linked list dengan data bertipe integer yang dapat melakukan operasi sisip secara terurut dan hapus simpul tertentu dengan ketentuan sebagai berikut :

- a. Operasi sisip, buat sebuah simpul baru,
 - Jika data simpul baru < data pada head, maka gunakan sisip awal list
 - Jika pencarian data simpul baru mencapai NULL (data belum ada), sisip akhir list
 - Jika data simpul baru = data pada simpul tertentu maka berikan pesan simpul sudah ada (duplikat)
 - Lainnya sisip sebelum simpul tertentu
- b. Operasi hapus,
 - Jika posisi simpul yang dihapus pada head, gunakan hapus awal list
 - Jika posisi simpul yang dihapus pada tail, gunakan hapus akhir list
 - Lainnya hapus simpul tengah

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. typedef struct simpul node;
5. struct simpul {
6.     int data;
7.     node *prev;
8.     node *next;
9. };
10.
11. node *head = NULL;
12. node *tail = NULL;
13.
14. node *buat_simpul(int);
15. void free_node(node *);
16. void sisip_terurut(int);
17. void hapus_simpul(int);
18. void tampilkan();
19.
20. int main() {
21.     char pilihan;
22.     int nilai;
23.
24.     do {
25.         printf("\noperasi double linked list (integer)\n");
26.         printf("a. sisip data terurut\n");
27.         printf("b. hapus data\n");
28.         printf("c. tampilkan list\n");
```

```

29.     printf("x. keluar\n");
30.     printf("pilih menu: ");
31.     scanf(" %c", &pilihan);
32.
33.     if (pilihan == 'x' || pilihan == 'X') {
34.         printf("keluar dari program\n");
35.         break;
36.     }
37.
38.     switch (pilihan) {
39.         case 'A':
40.         case 'a':
41.             printf("masukkan angka yang disisipkan: ");
42.             scanf("%d", &nilai);
43.             sisip_terurut(nilai);
44.             break;
45.         case 'B':
46.         case 'b':
47.             printf("masukkan angka yang dihapus: ");
48.             scanf("%d", &nilai);
49.             hapus_simpul(nilai);
50.             break;
51.         case 'C':
52.         case 'c':
53.             tampilkan();
54.             break;
55.         default:
56.             printf("pilihan tidak valid!\n");
57.     }
58. } while (1);
59.
60. return 0;
61.}
62.
63.node *buat_simpul(int x) {
64.     node *baru = (node *) malloc(sizeof(node));
65.     baru->data = x;
66.     baru->prev = NULL;
67.     baru->next = NULL;
68.     return baru;
69.}
70.
71.void free_node(node *p) {

```

```

72.     free(p);
73.     p = NULL;
74. }
75.
76. void sisip_terurut(int x) {
77.     node *baru = buat_simpul(x);
78.
79.     if (head == NULL) {
80.         head = tail = baru;
81.         printf("berhasil menyisipkan %d\n", x);
82.         return;
83.     }
84.
85.     node *temp = head;
86.     while (temp != NULL) {
87.         if (temp->data == x) {
88.             printf("simpul sudah ada (duplikat)!\n");
89.             free_node(baru);
90.             return;
91.         }
92.         temp = temp->next;
93.     }
94.
95.     if (x < head->data) {
96.         baru->next = head;
97.         head->prev = baru;
98.         head = baru;
99.         printf("berhasil menyisip awal: %d\n", x);
100.        return;
101.    }
102.
103.    node *current = head;
104.    while (current != NULL && current->data < x) {
105.        current = current->next;
106.    }
107.
108.    if (current == NULL) {
109.        tail->next = baru;
110.        baru->prev = tail;
111.        tail = baru;
112.        printf("berhasil menyisip akhir: %d\n", x);
113.    } else {
114.        baru->next = current;

```

```

115.         baru->prev = current->prev;
116.         current->prev->next = baru;
117.         current->prev = baru;
118.         printf("berhasil menyisip %d sebelum %d\n", x,
current->data);
119.     }
120. }
121.
122. void hapus_simpul(int x) {
123.     if (head == NULL) {
124.         printf("list kosong!\n");
125.         return;
126.     }
127.
128.     node *current = head;
129.     while (current != NULL && current->data != x) {
130.         current = current->next;
131.     }
132.
133.     if (current == NULL) {
134.         printf("simpul tidak ada\n");
135.         return;
136.     }
137.
138.     if (current == head) {
139.         head = head->next;
140.         if (head != NULL) head->prev = NULL;
141.         else tail = NULL;
142.         printf("berhasil hapus awal: %d\n", x);
143.     } else if (current == tail) {
144.         tail = tail->prev;
145.         tail->next = NULL;
146.         printf("berhasil hapus akhir: %d\n", x);
147.     } else {
148.         current->prev->next = current->next;
149.         current->next->prev = current->prev;
150.         printf("berhasil hapus tengah: %d\n", x);
151.     }
152.
153.     free_node(current);
154. }
155.
156. void tampilkan() {

```

```

157.         node *temp = head;
158.         printf("isi list: ");
159.         while (temp != NULL) {
160.             printf("%d <-> ", temp->data);
161.             temp = temp->next;
162.         }
163.         printf("null\n");
164.     }

```

Output:

```

operasi double linked list (integer)
a. sisip data terurut
b. hapus data
c. tampilkan list
x. keluar
pilih menu: c
isi list: null

```

```

operasi double linked list (integer)
a. sisip data terurut
b. hapus data
c. tampilkan list
x. keluar
pilih menu: a
masukkan angka yang disisipkan: 30
berhasil menyisipkan 30

```

```

operasi double linked list (integer)
a. sisip data terurut
b. hapus data
c. tampilkan list
x. keluar
pilih menu: a
masukkan angka yang disisipkan: 10
berhasil menyisip awal: 10

```

```

operasi double linked list (integer)
a. sisip data terurut
b. hapus data
c. tampilkan list
x. keluar
pilih menu: a
masukkan angka yang disisipkan: 50
berhasil menyisip akhir: 50

```

```

operasi double linked list (integer)
a. sisip data terurut
b. hapus data
c. tampilkan list
x. keluar
pilih menu: a
masukkan angka yang disisipkan: 20
berhasil menyisip 20 sebelum 30

```

```

operasi double linked list (integer)
a. sisip data terurut
b. hapus data
c. tampilkan list
x. keluar
pilih menu: a
masukkan angka yang disisipkan: 30
simpul sudah ada (duplikat)!

```

<pre> operasi double linked list (integer) a. sisip data terurut b. hapus data c. tampilkan list x. keluar pilih menu: c isi list: 10 <-> 20 <-> 30 <-> 50 <-> operasi double linked list (integer) a. sisip data terurut b. hapus data c. tampilkan list x. keluar pilih menu: b masukkan angka yang dihapus: 10 berhasil hapus awal: 10 operasi double linked list (integer) a. sisip data terurut b. hapus data c. tampilkan list x. keluar pilih menu: b masukkan angka yang dihapus: 20 berhasil hapus awal: 20 </pre>	<pre> operasi double linked list (integer) a. sisip data terurut b. hapus data c. tampilkan list x. keluar pilih menu: b masukkan angka yang dihapus: 100 simpul tidak ada operasi double linked list (integer) a. sisip data terurut b. hapus data c. tampilkan list x. keluar pilih menu: c isi list: 30 <-> 50 <-> null operasi double linked list (integer) a. sisip data terurut b. hapus data c. tampilkan list x. keluar pilih menu: x keluar dari program </pre>
---	--

Analisis Program:

Program ini mendemonstrasikan implementasi dasar memori dinamis menggunakan Double Linked List dengan alur yang dimulai dari proses penyisipan data secara terurut, di mana algoritma melakukan penelusuran menggunakan pointer bantuan untuk membandingkan nilai dan memanipulasi dua arah tautan sekaligus (pointer next dan prev) agar rantai list tidak terputus saat menyambungkan data baru. Algoritma ini sekaligus menangani berbagai skenario khusus seperti list kosong, penyisipan di awal, penyisipan di akhir, hingga pemblokiran input duplikat. Hal ini kemudian dilanjutkan dengan operasi penghapusan data di mana program secara berurutan mencari lokasi target terlebih dahulu untuk kemudian menyambungkan kembali tautan pointer dua arah dari node sebelum target langsung ke node sesudahnya secara presisi, sehingga node target terlepas dari rantai secara aman dan dapat dikembalikan ke sistem memori menggunakan perintah free.

2. Implementasikan Queue dengan double linked list, cobalah untuk menyelesaikan permasalahan shift bilangan decimal seperti pada praktikum Queue

```

1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4.
5. typedef struct simpul node;
6. struct simpul {
7.     int data;
8.     node *prev;

```

```

9.     node *next;
10.};
11.
12.node *front = NULL;
13.node *rear = NULL;
14.
15.node *buat_simpul(int);
16.void free_node(node *);
17.void enqueue(int);
18.int dequeue();
19.void shift();
20.void tampilkan_queue();
21.
22.int main() {
23.    char pilihan;
24.    char desimal[50];
25.
26.    printf("masukkan bilangan desimal (cth: 12345): ");
27.    scanf("%s", desimal);
28.
29.    for (int i = 0; i < strlen(desimal); i++) {
30.        enqueue(desimal[i] - '0');
31.    }
32.
33.    do {
34.        printf("\noperasi queue shift bilangan\n");
35.        printf("a. tampilkan antrean (queue)\n");
36.        printf("b. lakukan shift (geser)\n");
37.        printf("x. keluar\n");
38.        printf("pilih menu: ");
39.        scanf(" %c", &pilihan);
40.
41.        if (pilihan == 'x' || pilihan == 'X') {
42.            printf("keluar dari program\n");
43.            break;
44.        }
45.
46.        switch (pilihan) {
47.            case 'A':
48.            case 'a':
49.                tampilkan_queue();
50.                break;
51.            case 'B':

```



```

52.         case 'b':
53.             shift();
54.             break;
55.         default:
56.             printf("pilihan tidak valid!\n");
57.     }
58. } while (1);
59.
60. return 0;
61.}
62.
63.node *buat_simpul(int x) {
64.    node *baru = (node *) malloc(sizeof(node));
65.    baru->data = x;
66.    baru->prev = NULL;
67.    baru->next = NULL;
68.    return baru;
69.}
70.
71.void free_node(node *p) {
72.    free(p);
73.    p = NULL;
74.}
75.
76.void enqueue(int x) {
77.    node *baru = buat_simpul(x);
78.    if (rear == NULL) {
79.        front = rear = baru;
80.        return;
81.    }
82.    rear->next = baru;
83.    baru->prev = rear;
84.    rear = baru;
85.}
86.
87.int dequeue() {
88.    if (front == NULL) return -1;
89.    node *hapus = front;
90.    int nilai = hapus->data;
91.
92.    front = front->next;
93.    if (front == NULL) {
94.        rear = NULL;

```

```

95.     } else {
96.         front->prev = NULL;
97.     }
98.
99.     free_node(hapus);
100.     return nilai;
101. }
102.
103. void shift() {
104.     if (front == NULL) {
105.         printf("queue kosong!\n");
106.         return;
107.     }
108.     int nilai = dequeue();
109.     enqueue(nilai);
110.     printf("shift berhasil dilakukan.\n");
111.     tampilkan_queue();
112. }
113.
114. void tampilkan_queue() {
115.     node *temp = front;
116.     printf("isi queue (depan ke belakang): ");
117.     while (temp != NULL) {
118.         printf("%d ", temp->data);
119.         temp = temp->next;
120.     }
121.     printf("\n");
122. }

```

Output:

```
masukkan bilangan desimal (cth: 12345): 12345

operasi queue shift bilangan
a. tampilkan antrean (queue)
b. lakukan shift (geser)
x. keluar
pilih menu: a
isi queue (depan ke belakang): 1 2 3 4 5

operasi queue shift bilangan
a. tampilkan antrean (queue)
b. lakukan shift (geser)
x. keluar
pilih menu: b
shift berhasil dilakukan.
isi queue (depan ke belakang): 2 3 4 5 1

operasi queue shift bilangan
a. tampilkan antrean (queue)
b. lakukan shift (geser)
x. keluar
pilih menu: a
isi queue (depan ke belakang): 3 4 5 1 2

operasi queue shift bilangan
a. tampilkan antrean (queue)
b. lakukan shift (geser)
x. keluar
pilih menu: b
shift berhasil dilakukan.
isi queue (depan ke belakang): 4 5 1 2 3

operasi queue shift bilangan
a. tampilkan antrean (queue)
b. lakukan shift (geser)
x. keluar
pilih menu: x
keluar dari program
```

Analisis Program:

Program ini membuktikan fleksibilitas Double Linked List yang dapat dikonstruksi menjadi struktur Abstract Data Type berupa Queue berkonsep First In First Out (FIFO) melalui pembatasan akses manipulasi data yang dikendalikan secara ketat pada dua titik akses utama, yakni front dan rear. Alur program memanfaatkan fungsi penyisipan akhir sebagai operasi Enqueue untuk memasukkan data dan fungsi hapus awal sebagai operasi Dequeue untuk mengeluarkan data demi menyelesaikan permasalahan rotasi (shift) bilangan desimal. Karakteristik alami pemanggilan Queue berbasis pointer ganda ini secara otomatis mampu melepas simpul terdepan dan menyambungkannya kembali ke bagian ekor antrean secara presisi tanpa memerlukan trik indeks perulangan statis maupun merusak stabilitas memori berantai di dalamnya.

3. Buatlah double linked list dengan simpul berupa data mahasiswa yang terdiri dari NRP, Nama dan Kelas. Buatlah operasi Sisip secara terurut berdasarkan NRP, Hapus data mahasiswa tertentu berdasarkan NRP dan Update data berdasarkan NRP (nama dan kelas saja)

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <string.h>
4.
5. typedef struct simpul node;
6. struct simpul {
```

```

7.     int nrp;
8.     char nama[50];
9.     char kelas[10];
10.    node *prev;
11.    node *next;
12.};
13.
14.node *head = NULL;
15.node *tail = NULL;
16.
17.node *buat_simpul(int, char*, char*);
18.void free_node(node *);
19.void sisip_mhs(int, char*, char*);
20.void hapus_mhs(int);
21.void update_mhs(int);
22.void tampilkan_mhs();
23.
24.int main() {
25.    char pilihan;
26.    int nrp;
27.    char nama[50], kelas[10];
28.
29.    do {
30.        printf("\noperasi data mahasiswa (double linked
list)\n");
31.        printf("a. sisip mahasiswa (terurut nrp)\n");
32.        printf("b. hapus mahasiswa (berdasar nrp)\n");
33.        printf("c. update mahasiswa (berdasar nrp)\n");
34.        printf("d. tampilkan data\n");
35.        printf("x. keluar\n");
36.        printf("pilih menu: ");
37.        scanf(" %c", &pilihan);
38.
39.        if (pilihan == 'x' || pilihan == 'X') {
40.            printf("keluar dari program\n");
41.            break;
42.        }
43.
44.        switch (pilihan) {
45.            case 'A':
46.            case 'a':
47.                printf("nrp: "); scanf("%d", &nrp);
48.                printf("nama: "); scanf(" %[^n]", nama);

```

```

49.         printf("kelas: "); scanf("%[^\\n]", kelas);
50.         sisip_mhs(nrp, nama, kelas);
51.         break;
52.     case 'B':
53.     case 'b':
54.         printf("masukkan nrp yang akan dihapus: ");
55.         scanf("%d", &nrp);
56.         hapus_mhs(nrp);
57.         break;
58.     case 'C':
59.     case 'c':
60.         printf("masukkan nrp yang akan diupdate: ");
61.         scanf("%d", &nrp);
62.         update_mhs(nrp);
63.         break;
64.     case 'D':
65.     case 'd':
66.         tampilkan_mhs();
67.         break;
68.     default:
69.         printf("pilihan tidak valid!\\n");
70.     }
71. } while (1);
72.
73. return 0;
74.}
75.
76.node *buat_simpul(int nrp, char *nama, char *kelas) {
77.    node *baru = (node *) malloc(sizeof(node));
78.    baru->nrp = nrp;
79.    strcpy(baru->nama, nama);
80.    strcpy(baru->kelas, kelas);
81.    baru->prev = NULL;
82.    baru->next = NULL;
83.    return baru;
84.}
85.
86.void free_node(node *p) {
87.    free(p);
88.    p = NULL;
89.}
90.
91.void sisip_mhs(int nrp, char *nama, char *kelas) {

```

```

92.     node *baru = buat_simpul(nrp, nama, kelas);
93.
94.     if (head == NULL) {
95.         head = tail = baru;
96.         printf("data pertama berhasil ditambahkan.\n");
97.         return;
98.     }
99.
100.    if (nrp < head->nrp) {
101.        baru->next = head;
102.        head->prev = baru;
103.        head = baru;
104.        printf("data berhasil ditambahkan di awal.\n");
105.        return;
106.    }
107.
108.    node *current = head;
109.    while (current != NULL && current->nrp < nrp) {
110.        if (current->nrp == nrp) {
111.            printf("nrp sudah ada!\n");
112.            free_node(baru);
113.            return;
114.        }
115.        current = current->next;
116.    }
117.
118.    if (current == NULL) {
119.        tail->next = baru;
120.        baru->prev = tail;
121.        tail = baru;
122.    } else {
123.        baru->next = current;
124.        baru->prev = current->prev;
125.        current->prev->next = baru;
126.        current->prev = baru;
127.    }
128.    printf("data mahasiswa berhasil disisipkan.\n");
129. }
130.
131. void hapus_mhs(int nrp) {
132.     if (head == NULL) return;
133.
134.     node *current = head;

```

```

135.         while (current != NULL && current->nrp != nrp) {
136.             current = current->next;
137.         }
138.
139.         if (current == NULL) {
140.             printf("mahasiswa dengan nrp %d tidak
ditemukan.\n", nrp);
141.             return;
142.         }
143.
144.         if (current == head) {
145.             head = head->next;
146.             if (head != NULL) head->prev = NULL;
147.             else tail = NULL;
148.         } else if (current == tail) {
149.             tail = tail->prev;
150.             tail->next = NULL;
151.         } else {
152.             current->prev->next = current->next;
153.             current->next->prev = current->prev;
154.         }
155.
156.         free_node(current);
157.         printf("data mahasiswa nrp %d berhasil dihapus.\n",
nrp);
158.     }
159.
160.     void update_mhs(int nrp) {
161.         node *current = head;
162.         while (current != NULL) {
163.             if (current->nrp == nrp) {
164.                 printf("masukkan nama baru: ");
165.                 scanf(" %[^\\n]", current->nama);
166.                 printf("masukkan kelas baru: ");
167.                 scanf(" %[^\\n]", current->kelas);
168.                 printf("data mahasiswa berhasil
diupdate!\\n");
169.                 return;
170.             }
171.             current = current->next;
172.         }
173.         printf("mahasiswa dengan nrp %d tidak ditemukan.\n",
nrp);

```

```

174.     }
175.
176.     void tampilkan_mhs() {
177.         node *temp = head;
178.         if (temp == NULL) {
179.             printf("data mahasiswa kosong.\n");
180.             return;
181.         }
182.         printf("\n--- data mahasiswa ---\n");
183.         while (temp != NULL) {
184.             printf("nrp   : %d\n", temp->nrp);
185.             printf("nama   : %s\n", temp->nama);
186.             printf("kelas  : %s\n\n", temp->kelas);
187.             temp = temp->next;
188.         }
189.     }

```

Output:

operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: d
data mahasiswa kosong.

operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: a
nrp: 200
nama: budi
kelas: 1a
data pertama berhasil ditambahkan.

operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: a
nrp: 100
nama: andi
kelas: 1a
data berhasil ditambahkan di awal.

operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: a
nrp: 300
nama: citra
kelas: 1b
data mahasiswa berhasil disisipkan.

operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: a
nrp: 200
nama: joko
kelas: 1c
data mahasiswa berhasil disisipkan.


```
operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: d
```

```
--- data mahasiswa ---
```

```
nrp   : 100
nama  : andi
kelas : 1a
```

```
nrp   : 200
nama  : joko
kelas : 1c
```

```
nrp   : 200
nama  : budi
kelas : 1a
```

```
nrp   : 300
nama  : citra
kelas : 1b
```

```
operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
```

```
x. keluar
```

```
pilih menu: c
```

```
masukkan nrp yang akan diupdate: 200
```

```
masukkan nama baru: budi santoso
```

```
masukkan kelas baru: 2a
```

```
data mahasiswa berhasil diupdate!
```

```
operasi data mahasiswa (double linked list)
```

```
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
```

```
x. keluar
```

```
pilih menu: c
```

```
masukkan nrp yang akan diupdate: 999
```

```
mahasiswa dengan nrp 999 tidak ditemukan.
```

```
operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: d
```

```
--- data mahasiswa ---
```

```
nrp   : 100
nama  : andi
kelas : 1a
```

```
nrp   : 200
nama  : budi santoso
kelas : 2a
```

```
nrp   : 200
nama  : budi
kelas : 1a
```

```
nrp   : 300
nama  : citra
kelas : 1b
```

```
operasi data mahasiswa (double linked list)
```

```
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
```

```
x. keluar
```

```
pilih menu: b
```

```
masukkan nrp yang akan dihapus: 100
```

```
data mahasiswa nrp 100 berhasil dihapus.
```

```
operasi data mahasiswa (double linked list)
```

```
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
```

```
x. keluar
```

```
pilih menu: b
```

```
masukkan nrp yang akan dihapus: 300
```

```
data mahasiswa nrp 300 berhasil dihapus.
```

```

operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: d

--- data mahasiswa ---
nrp   : 200
nama  : budi santoso
kelas : 2a

nrp   : 200
nama  : budi
kelas : 1a

operasi data mahasiswa (double linked list)
a. sisip mahasiswa (terurut nrp)
b. hapus mahasiswa (berdasar nrp)
c. update mahasiswa (berdasar nrp)
d. tampilkan data
x. keluar
pilih menu: x
keluar dari program

```

Analisis Program:

Program terakhir ini membuat Double Linked List yang lebih kompleks untuk mengelola tipe data bentukan majemuk dengan cara menyatukan entitas rekaman mahasiswa yang mencakup NRP bertipe integer sebagai acuan parameter pengurutan beserta atribut tambahan berupa string nama dan kelas ke dalam satu node utuh. Alur programnya diawali dengan mengurutkan data masuk murni berdasarkan perbandingan angka NRP dengan menautkan rantai dua arah, dilanjutkan dengan kemampuan menyediakan fitur pembaruan (update) yang mendemonstrasikan bahwa manipulasi list tidak terbatas pada pembongkaran rute pointer saja, melainkan mampu melakukan penimpaan konten teks menggunakan fungsi penyalinan string langsung pada alamat memori node spesifik yang dicari tanpa merusak kestabilan susunan rantai pointer ganda di sekitarnya.

Kesimpulan

Pelaksanaan praktikum Algoritma dan Struktur Data mengenai Double Linked List (DLL) ini secara komprehensif membuktikan bahwa arsitektur alokasi memori dinamis dengan navigasi dua arah jauh lebih efisien dan tangguh dibandingkan array statis maupun Single Linked List karena ruang memori hanya dibentuk secara presisi saat dibutuhkan dan langsung dikembalikan ke sistem saat tidak terpakai. Pengelolaannya menuntut ketelitian tinggi dalam menggunakan pointer bantuan penelusur data untuk menautkan dan memutus jangkar next dan prev tanpa menggeser jangkar utama penunjuk awal (head) dan akhir (tail) agar rantai memori tidak terputus saat mengeksekusi operasi sisip terurut maupun hapus.

Struktur dasar ini juga menunjukkan keandalannya saat diterapkan untuk memecahkan masalah komputasi terstruktur sebagai fondasi pembentukan Queue berkonsep First In First Out (FIFO) untuk menyelesaikan pergeseran (shift)

susunan angka. Hingga pada akhirnya berhasil diekspansi menjadi purwarupa database yang sanggup mengelola rekaman majemuk entitas mahasiswa untuk menjaga keterurutan data berdasarkan nomor identitas sekaligus mengimplementasikan fitur pembaruan konten teks di dalam simpul secara langsung tanpa perlu merusak susunan tautan pointer ganda pada keseluruhan sistem senarai berantai tersebut.